

SIMATS ENGINEERING ASSESSMENT TOOL 2

COURSE CODE: CSA5112

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.* CO (BL4 , BL5)

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A CASE STUDY

Code of Conduct:

I K. Bhavana (192524178) _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

K.Bhavana

Annexure A

1. Weak Password Hashing in Web Application

A web application stores user passwords using unsalted SHA-1 hashes. After a breach, attackers quickly crack multiple passwords using precomputed tables.

Analyze how the lack of salting and use of SHA-1 contributed to this vulnerability.

Recommend a secure password hashing strategy and justify your choice.

2. Digital Signature Compromise in Financial System

A financial institution uses digital signatures for transaction approval. An attacker gains access to a user's private key and performs unauthorized transactions.

Evaluate how the compromise affects authentication and non-repudiation. Propose mitigation strategies to secure private keys and prevent such incidents.

3. Kerberos Authentication Failure in Distributed System

A distributed system using Kerberos experiences frequent authentication failures due to time synchronization issues between servers.

Analyze how time synchronization impacts Kerberos functionality. Suggest solutions to ensure reliable authentication and maintain system security.

4. Certificate Authority Breach in PKI System

A Certificate Authority (CA) is compromised, leading to the issuance of fake X.509 certificates for trusted domains.

Evaluate the impact of this breach on system trust and secure communication. Propose mechanisms to detect and mitigate such attacks in a PKI environment.

5. Integrity Verification in Cloud Data Storage

A cloud service provider uses hashing to verify file integrity, but users report data tampering incidents. Investigation reveals improper hash validation practices.

Analyze how incorrect implementation of hashing can lead to integrity failures.

Recommend a robust integrity verification mechanism and justify its effectiveness.

Question 1: Weak Password Hashing in Web Application

1.1 Case Scenario

A web application stores user passwords using unsalted SHA-1 hashes. Following a data breach, attackers are able to crack a large number of passwords quickly using precomputed lookup tables. This case requires an analysis of why the combination of SHA-1 and the absence of salting created this vulnerability, and a justified recommendation for a secure replacement.

1.2 Why SHA-1 Is Unsuitable for Password Storage

SHA-1 is a general-purpose cryptographic hash function that was designed for speed and for verifying data integrity, not for protecting secrets such as passwords. Its design goals work directly against password security for the following reasons:

- Computational speed: SHA-1 can be computed at billions of hashes per second on commodity GPUs. A defender wants password verification to be slow for an attacker; SHA-1 is fast, which is the opposite of what is needed.
- Hardware acceleration: SHA-1 is simple enough to implement efficiently in ASICs and FPGAs, letting attackers scale brute-force and dictionary attacks massively and cheaply.
- Cryptanalytic weakness: SHA-1 collision attacks (e.g., the 2017 'SHattered' demonstration) proved that two different inputs can be crafted to produce the same digest. While collision resistance is not the primary issue for password cracking (which relies on preimage-style guessing), it confirms SHA-1 no longer meets modern cryptographic assurance levels and has been deprecated by NIST for security-critical uses.
- No configurable work factor: SHA-1 has no built-in mechanism to slow down verification as hardware improves, unlike password-specific functions.

1.3 Why the Absence of Salting Made the Breach Worse

A salt is random, unique data appended to each password before hashing. Without a salt, the vulnerability compounds in three specific ways:

- Precomputed (rainbow) table attacks: An attacker can precompute hashes for billions of common passwords once and reuse that table against every stolen hash in the database, cracking many accounts almost instantly.
- Identical passwords produce identical hashes: Attackers can immediately see which users share the same password, and cracking one hash cracks every account using that password simultaneously.
- Cross-database correlation: Because unsalted SHA-1 of a given password is always the same value, attackers can match hashes in this breach against hashes leaked from unrelated breaches, defeating any secrecy the application believed it had.

In short, the breach was not caused by a single flaw but by the compounding of two weaknesses: a hash function optimized for speed, and the absence of the one mechanism (salting) that would have forced attackers to attack each password individually.

1.4 Comparison of Password Hashing Approaches

Scheme	Design Goal	Salting	Work Factor	Memory-Hard	Suitability for Passwords
Unsalted SHA-1	General-purpose integrity hashing	None	Fixed / fast	No	Unsuitable — vulnerable to rainbow tables and GPU cracking
Salted SHA-256	General-purpose hashing + salt	Per-user	Fixed / fast	No	Better than SHA-1, but still too fast; brute-forceable at scale
bcrypt	Password hashing (Blowfish-based)	Built-in, automatic	Adjustable (cost factor)	No	Good — widely adopted, tunable, resistant to simple GPU scaling
scrypt	Password hashing / KDF	Built-in	Adjustable	Yes (moderate)	Strong — memory-hardness raises hardware attack cost
Argon2id	Password Hashing Competition winner (2015)	Built-in, mandatory	Adjustable (time, memory, parallelism)	Yes (tunable)	Best current choice — resistant to GPU/ASIC and side-channel attacks

1.5 Recommended Strategy and Justification

The recommended approach is to migrate to Argon2id, using a unique cryptographically random salt per password (at least 16 bytes), combined with the following implementation practices:

- Use Argon2id specifically (not Argon2d or Argon2i alone) because it combines resistance to GPU/side-channel attacks with resistance to time-memory trade-off attacks, making it the current industry and OWASP-recommended default.
- Tune memory, iteration, and parallelism parameters to the highest cost the application's hardware budget can tolerate (OWASP recommends a minimum of 19 MiB memory, 2 iterations, 1 degree of parallelism as a floor, higher where feasible).
- Never truncate or reuse salts; generate a new salt with a cryptographically secure random number generator for every password.
- Apply an additional server-side 'pepper' (a secret key stored outside the database, e.g., in an HSM or secrets manager) as defense-in-depth so a database-only breach does not immediately expose crackable material.
- Enforce a rehash-on-login migration path: when a legacy SHA-1 user logs in successfully, transparently rehash and store their password using Argon2id, retiring SHA-1 gradually without forcing a mass password reset.

- Add rate limiting, account lockout, and breach-monitoring (e.g., checking against known-compromised password lists) as complementary layers, since hashing strength alone cannot stop credential-stuffing attacks.

This strategy is justified because Argon2id directly counters the two root causes identified: it is deliberately slow and tunable (countering SHA-1's speed problem) and it mandates salting (countering the precomputed-table problem), while its memory-hardness further neutralizes the economic advantage attackers gain from GPUs and ASICs.

Question 2: Digital Signature Compromise in Financial System

2.1 Case Scenario

A financial institution relies on digital signatures to approve transactions. An attacker obtains a user's private key and uses it to perform unauthorized transactions. This case requires evaluating the impact on authentication and non-repudiation, and proposing mitigation strategies to protect private keys.

2.2 How Digital Signatures Provide Authentication and Non-Repudiation

A digital signature scheme (typically RSA or ECDSA combined with a hash function) works as follows: the signer hashes the transaction data and encrypts the hash using their private key to produce a signature; anyone holding the corresponding public key can verify that the signature matches the data. Security rests entirely on one assumption: that only the legitimate owner possesses the private key. Under that assumption, a valid signature proves two properties simultaneously — authentication (the message came from the key holder) and non-repudiation (the signer cannot later deny having signed it, since only they could have produced that signature).

2.3 Impact of the Private Key Compromise

- **Loss of authentication:** Once the attacker holds the private key, the system can no longer distinguish attacker-signed transactions from genuine ones — the cryptographic proof of identity is broken at its foundation.
- **Loss of non-repudiation:** The core guarantee that 'only the account holder could have produced this signature' is violated. Ironically, this cuts both ways — the attacker's fraudulent transactions appear legitimate, while the legitimate user may later be able to plausibly deny genuine transactions by pointing to the known compromise, undermining legal enforceability of the signature record.
- **Financial and reputational damage:** Unauthorized transactions can be executed and settled before detection, and once trust in the signing mechanism is broken, the institution may need to invalidate and reissue keys for the affected user, and potentially audit all transactions signed under that key.
- **Cascading trust failure:** If the key is tied to a certificate issued by the institution's own PKI, the compromise may also require certificate revocation, which introduces operational disruption for the legitimate user.

2.4 Mitigation Strategies to Secure Private Keys

Attack Vector	Mitigation Strategy	How It Helps
Key stored as a software file / in memory	Hardware Security Module (HSM) or smart card / secure enclave storage	Private key material never leaves tamper-resistant hardware; signing occurs inside the device
Phishing / credential theft leading to key export	Multi-factor authentication before any signing operation	Even a stolen key or credential cannot be used without a second, independent factor

Attack Vector	Mitigation Strategy	How It Helps
Single point of failure (one key, one signer)	Threshold signatures / multi-signature (m-of-n) schemes	No single compromised key is sufficient to authorize a high-value transaction
Long key lifetime increases exposure window	Short-lived keys / certificates with automatic rotation	Limits the time window an attacker can exploit a stolen key before it expires
Delayed detection of misuse	Real-time transaction monitoring and anomaly detection	Unusual transaction patterns trigger holds or step-up verification before funds move
No way to invalidate a stolen key quickly	Certificate Revocation Lists (CRL) / OCSP with fast propagation	Compromised keys can be revoked and rejected by verifiers almost immediately
Human error / social engineering	Security awareness training and least-privilege key access	Reduces the likelihood of key material being exposed in the first place

2.5 Additional Preventive and Recovery Measures

- Maintain an out-of-band, pre-agreed incident response plan: on suspected compromise, immediately revoke the certificate, notify the CA, freeze affected accounts, and reissue new key pairs generated inside a fresh HSM.
- Apply trusted timestamping to signatures so that the time of signing is cryptographically bound and independently verifiable, strengthening evidentiary value even in disputed cases.
- Segregate signing duties for high-value transactions (dual control) so that no single compromised identity can authorize a transaction alone.
- Maintain immutable audit logs of all signing operations, ideally write-once or blockchain-anchored, to support forensic investigation.

Together, these controls shift private key protection from a single point of trust to a defense-in-depth model, ensuring that the compromise of one credential is not sufficient on its own to authorize fraud, and that recovery can happen quickly when a compromise is detected.

Question 3: Kerberos Authentication Failure in Distributed System

3.1 Case Scenario

A distributed system using Kerberos experiences frequent authentication failures caused by time synchronization issues between servers. This case requires analyzing why clock skew affects Kerberos and proposing solutions for reliable, secure authentication.

3.2 Why Kerberos Depends on Synchronized Clocks

Kerberos authentication is built around time-limited tickets (the Ticket Granting Ticket and Service Tickets) and Authenticators that include a timestamp. This design exists specifically to prevent replay attacks: if an attacker intercepts a valid ticket or authenticator, they cannot reuse it indefinitely because it expires. Because the Key Distribution Centre (KDC) and every client/server in the realm independently check timestamps against their own local clock, all machines must agree closely on the current time for this mechanism to work. Standard Kerberos implementations (e.g., MIT Kerberos, Microsoft Active Directory) enforce a maximum clock skew — commonly a default of five minutes — beyond which tickets are rejected outright, regardless of whether they are otherwise valid.

3.3 How Time Desynchronization Causes Authentication Failures

- Ticket rejection: If a client's clock drifts beyond the allowed skew relative to the KDC, the KDC rejects the authentication request with an error such as 'Clock skew too great', even though the credentials themselves are correct.
- Replay window manipulation: The skew tolerance also defines the window within which a captured authenticator could theoretically be replayed. If skew is not tightly controlled, this window widens, weakening the anti-replay protection Kerberos is meant to provide.
- Cascading failures in distributed systems: In a multi-server distributed environment, if even one node's clock drifts (due to a faulty NTP client, a VM losing time sync after a pause/resume, or manual misconfiguration), that node fails to authenticate with every other synchronized node, producing failures that appear intermittent and difficult to diagnose.
- Security risk if skew tolerance is loosened carelessly: A common but risky 'fix' is to increase the allowed skew tolerance to reduce failures; this directly weakens replay-attack protection and should not be used as the primary solution.

3.4 Root Causes and Corresponding Solutions

Symptom	Likely Root Cause	Recommended Solution
Intermittent 'Clock skew too great' errors	No centralized, authoritative time source	Deploy an internal NTP hierarchy synchronized to a reliable stratum-1 source
Failures after VM snapshot/resume	Virtual machine clock drift	Enable hypervisor time synchronization and periodic NTP resync on VM boot

Symptom	Likely Root Cause	Recommended Solution
Failures concentrated on specific hosts	Misconfigured or missing NTP client on those hosts	Audit and enforce NTP/chrony configuration via centralized configuration management
Skew errors despite NTP running	Unauthenticated NTP traffic spoofed or blocked by network devices	Adopt NTP with authentication, or Network Time Security (NTS), and monitor for spoofed responses
Failures across time zones/regions	Incorrect timezone configuration rather than true clock drift	Standardize all servers to UTC internally, converting only for display
No visibility into drift before failures occur	Lack of monitoring	Implement drift monitoring and alerting (e.g., threshold alerts before drift approaches the skew limit)

3.5 Recommendations for a Reliable Long-Term Solution

- Establish a redundant, hierarchical NTP infrastructure with multiple upstream time sources to avoid a single point of failure.
- Keep the Kerberos clock skew tolerance at its secure default (typically five minutes) rather than loosening it, and instead fix the underlying synchronization problem.
- Use authenticated time synchronization (NTP with symmetric keys, or NTS) to prevent an attacker from spoofing time responses to artificially widen the replay window.
- Automate periodic time-sync verification and alerting so drift is caught and corrected before it causes authentication failures.
- For particularly high-precision environments, consider Precision Time Protocol (PTP) instead of NTP, though NTP is sufficient for the tolerance Kerberos requires.

Correcting the root time-synchronization problem — rather than relaxing Kerberos's skew tolerance — preserves both system reliability and the replay-attack protection that timestamped tickets are designed to provide.

Question 4: Certificate Authority Breach in PKI System

4.1 Case Scenario

A Certificate Authority (CA) is compromised, resulting in the issuance of fraudulent X.509 certificates for trusted domains. This case requires evaluating the impact on system-wide trust and secure communication, and proposing mechanisms to detect and mitigate such breaches.

4.2 Why a CA Breach Is Catastrophic for PKI Trust

Public Key Infrastructure relies on a hierarchical chain of trust: browsers and operating systems trust a small set of root CAs, and by extension trust any certificate that chains back to one of those roots. When a CA's signing infrastructure is compromised, the attacker gains the ability to mint certificates for any domain that will be accepted as valid by every relying party that trusts that CA — without needing to compromise the actual domain owner. This breaks both core guarantees of TLS: authentication (a client can no longer be sure it is talking to the real server) and confidentiality (a fraudulent certificate enables transparent man-in-the-middle interception, since the attacker can present a certificate that passes standard validation).

A well-known real-world illustration is the 2011 breach of the CA DigiNotar, in which attackers issued fraudulent certificates (including for major domains) that were subsequently used to intercept traffic; the incident ultimately led browsers and operating systems to permanently remove DigiNotar from their trusted root stores, effectively ending the CA's operations. This demonstrates that the consequences of a CA breach extend beyond the immediate victims to the CA's entire customer base and the broader trust ecosystem.

4.3 Impact Analysis

- Impersonation of trusted domains: Attackers can perform undetectable man-in-the-middle attacks against users of banking, email, or government sites, since the fraudulent certificate is cryptographically valid.
- Erosion of the trust model itself: Every relying party that trusts the breached CA is affected, not just the specific domains targeted, because the compromise proves the CA's issuance process cannot be relied upon.
- Delayed detection: Traditional PKI validation gives no visibility into unauthorized issuance, since a maliciously issued certificate passes the same checks as a legitimate one — the breach may go unnoticed until independent monitoring catches it.
- Business and regulatory consequences: Affected domain owners may suffer reputational and financial harm; the CA itself risks removal from trust stores, which is an existential business consequence.

4.4 Detection and Mitigation Mechanisms

Threat / Gap	Detection Mechanism	Mitigation Mechanism
Unauthorized certificate issuance goes unnoticed	Certificate Transparency (CT) logs — publicly auditable, append-only logs of all issued certificates	Domain owners run CT monitoring services to receive alerts when a certificate for their domain is logged
Any CA can issue a certificate for any domain	CAA (Certification Authority Authorization) DNS records advertise which CAs are permitted to issue for a domain	CAs are required to check CAA records before issuance, blocking unauthorized issuers
Compromised certificates remain trusted after discovery	Revocation status checks	OCSP stapling / CRLs allow relying parties to reject revoked certificates quickly
Root/intermediate signing keys are a single point of failure	Regular security audits (e.g., WebTrust audits) of CA operations	Store CA private keys in HSMs, keep root CA offline, use m-of-n multi-party control for signing ceremonies
Long-lived certificates extend attacker advantage	Certificate lifetime monitoring	Adopt shorter certificate validity periods to reduce the exposure window of any single compromise
Compromise not contained after detection	Incident response and forensic audit trail	Immediate revocation of fraudulent certificates and, if warranted, removal of the CA from root trust stores

4.5 Recommendations

- Mandate Certificate Transparency logging industry-wide (already required by major browsers) so that fraudulent issuance becomes publicly and rapidly detectable.
- Encourage domain owners to deploy CAA records restricting issuance to their chosen CA(s), sharply reducing the attack surface of a rogue or breached CA.
- Enforce strict operational security for CA infrastructure: offline root CAs, HSM-protected keys, multi-person authorization for signing operations, and regular independent audits.
- Support rapid, well-tested revocation infrastructure (OCSP with stapling) so that once fraudulent certificates are identified, clients stop trusting them quickly rather than relying on slow-to-propagate CRLs.
- Maintain an incident response plan that includes coordination with browser/OS vendors, since ultimately trust-store removal is the strongest deterrent and final safeguard against a negligent or repeatedly compromised CA.

Question 5: Integrity Verification in Cloud Data Storage

5.1 Case Scenario

A cloud service provider uses hashing to verify file integrity, but users report data tampering incidents. Investigation reveals improper hash validation practices. This case requires analyzing how incorrect hashing implementation causes integrity failures and recommending a robust verification mechanism.

5.2 How Improper Hashing Implementation Undermines Integrity

Hashing can only detect tampering if it is implemented so that an attacker who can modify the data cannot also regenerate a matching hash. Several common implementation mistakes break this assumption:

- Storing the hash alongside the data it protects, in the same mutable, attacker-accessible location: if an attacker can modify the file, they can simply recompute the (unkeyed) hash and overwrite the stored value, so tampering goes undetected.
- Using a plain, unkeyed hash (such as a bare SHA-256 digest) instead of a keyed construction: an unkeyed hash proves only that data matches a value, not that the value came from a trusted party — anyone can compute it, including an attacker.
- Using weak or deprecated hash functions (MD5, SHA-1) that are vulnerable to collision attacks, allowing a skilled attacker to craft tampered data that still matches a previously recorded hash.
- Validating integrity only at upload time and never again, so tampering that occurs later in storage is never detected.
- No mechanism for partial verification: for very large files, recomputing a single hash over the entire object on every check is inefficient, which in practice leads providers to skip regular verification altogether.

5.3 Recommended Robust Integrity Verification Mechanism

- Use a keyed hash (HMAC-SHA256 or stronger) with a secret key held only by the provider, or better, digital signatures over the file hash using the provider's private key — this binds integrity to authenticity, so an attacker who can alter stored data still cannot produce a valid HMAC/signature without the secret key.
- Adopt Merkle tree hashing for large or chunked files: each block is hashed individually, and hashes are combined hierarchically into a single root hash. This allows efficient verification of any individual chunk without rehashing the entire file, and makes tampering in any single block immediately detectable and localizable.
- Store the reference hash/signature separately from the data itself — for example, in an independent, access-controlled metadata store, a write-once log, or a distributed ledger — so that compromising the data store alone is not sufficient to defeat verification.
- Perform periodic, automated integrity audits (a form of Proof of Data Possession / Proof of Retrievability) rather than relying solely on validation at upload time, so tampering that occurs during storage is caught proactively.
- Apply layered checksums: transport-level integrity via TLS, storage-level checksums for hardware-level corruption, and application-level cryptographic hashing/signing for security-relevant tamper detection — each layer catches a different class of failure.
- Maintain immutable, timestamped audit logs of all integrity checks and any detected discrepancies to support forensic investigation and user notification.

5.4 Comparison of Integrity Verification Approaches

Approach	Detects Tampering?	Detects Unauthorized Regeneration?	Efficient for Large Files	Recommended Use
Plain unkeyed hash stored with data	Only if attacker lacks write access	No — attacker can recompute	Poor (full rehash)	Not recommended for untrusted storage
Plain hash stored separately	Yes, if separation is enforced	No — still unkeyed	Poor	Minimal improvement only
HMAC with provider secret key	Yes	Yes — key required to forge	Poor for very large files	Good for moderate file sizes
Digital signature over file hash	Yes	Yes — private key required	Poor alone	Strong authenticity + integrity binding
Merkle tree + signed root hash	Yes, and localizes tampered block	Yes	Efficient (partial verification)	Recommended for large/chunked cloud files

5.5 Justification of Effectiveness

A Merkle-tree-based scheme with a digitally signed root hash addresses every weakness identified in the original implementation. Because each leaf hash is combined cryptographically into a signed root, an attacker who modifies even a single byte of a single chunk changes that chunk's hash, which propagates up and changes the root hash — making the tampering mathematically detectable without needing to compare the entire file. Because the root hash is signed with the provider's private key rather than stored as a bare value, an attacker with write access to the storage layer cannot forge a matching signature, closing the exact gap that caused the original tampering incidents to go unnoticed. Combined with periodic automated audits and layered checksums, this approach provides continuous, efficient, and cryptographically sound assurance of data integrity in the cloud environment.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided

Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand
---------	----	--	---	-------------------------------	---------------------------------------

